

Pandas

June 9, 2024

0.1 ¿Qué es Pandas y para que sirve?

Pandas es una biblioteca de Python que se utiliza para el análisis y la manipulación de datos en bruto. Es una herramienta poderosa que puede ayudar a los usuarios a limpiar, transformar y analizar datos de una manera rápida y eficiente.

La estructura de datos principal en Pandas se llama DataFrame. Es una tabla de datos bidimensional que se compone de filas y columnas, y se asemeja a una hoja de cálculo de Excel. Los DataFrames son útiles para almacenar y manipular grandes cantidades de datos y proporcionan una gran cantidad de funcionalidades, como la selección y filtrado de datos, la agregación de datos y la realización de operaciones matemáticas y estadísticas.

0.2 Poniendo a punto Pandas

1. Importamos pandas con el alias `pd`.

```
[ ]: import pandas as pd
```

2. Mostramos la version de pandas que importamos.

```
[ ]: pd.__version__
```

```
[ ]: '2.2.2'
```

3. Imaginemos que tenemos el siguiente diccionario `data` y la lista `labels`:

```
data = {'animal': ['cat', 'cat', 'snake', 'dog', 'dog', 'cat', 'snake', 'cat', 'dog', 'dog'],  
        'age': [2.5, 3, 0.5, None, 5, 2, 4.5, None, 7, 3],  
        'visits': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],  
        'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no', 'no']}
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

Vamos a crear un DataFrame `df` a partir del diccionario `data` usando `labels` como índice.

```
[ ]: data = {'animal': ['cat', 'cat', 'snake', 'dog', 'dog', 'cat', 'snake', 'cat', 'dog',  
    ↪ 'dog'],  
        'age': [2.5, 3, 0.5, None, 5, 2, 4.5, None, 7, 3],  
        'visits': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],  
        'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no',  
    ↪ 'no']}
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

df = pd.DataFrame(data, index=labels)
df # Esto lo ponemos para mostrarlo
```

```
[ ]:  animal  age  visits  priority
a    cat   2.5      1      yes
b    cat   3.0      3      yes
c  snake   0.5      2      no
d    dog   NaN      3      yes
e    dog   5.0      2      no
f    cat   2.0      3      no
g  snake   4.5      1      no
h    cat   NaN      1      yes
i    dog   7.0      2      no
j    dog   3.0      1      no
```

4. Mostramos informacion que resuma al DataFrame

`df.info()` nos devuelve informacion sobre el DataFrame incluidos sus índices y columnas, valores no nulos y uso de memoria

```
[ ]: df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 10 entries, a to j
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   animal      10 non-null      object
1   age          8 non-null       float64
2   visits      10 non-null      int64
3   priority    10 non-null      object
dtypes: float64(1), int64(1), object(2)
memory usage: 400.0+ bytes
```

Otra forma de obtener algun tipo de informacion del DataFrame es utilizando la funcion **describe** la cual devuelve una serie con un resumen descriptivo que incluye el número de datos, su suma, el mínimo, el máximo, la media, la desviación típica y los cuartile

```
[ ]: df.describe() # Otra forma
```

```
[ ]:
count      age      visits
mean    3.437500    1.900000
std     2.007797    0.875595
min     0.500000    1.000000
25%     2.375000    1.000000
```

50%	3.000000	2.000000
75%	4.625000	2.750000
max	7.000000	3.000000

5. `df.head(n)` Devuelve las primeras `n` filas del DataFrame `df` En el caso de no especificarlo, nos devuelve las primeras 5. Mostramos las primeras 3 filas del DataFrame `df`.

```
[ ]: df.head(3)
```

```
[ ]:  animal  age  visits  priority
a     cat   2.5      1      yes
b     cat   3.0      3      yes
c  snake   0.5      2      no
```

```
[ ]: # Otra forma
df.iloc[:3]
```

```
[ ]:  animal  age  visits  priority
a     cat   2.5      1      yes
b     cat   3.0      3      yes
c  snake   0.5      2      no
```

6. Si queremos solamente mostrar las columnas 'animal' y 'age' columns del DataFrame `df`.

Vamos a utilizar `df.loc[filas, columnas]` lo que devuelve un DataFrame con los elementos de las filas de la lista `filas` y las columnas de la lista `columnas`. Si queremos que muestre todas las columnas podemos poner `:` en su lugar.

```
[ ]: df.loc[:, ['animal', 'age']]
```

```
[ ]:  animal  age
a     cat   2.5
b     cat   3.0
c  snake   0.5
d     dog  NaN
e     dog   5.0
f     cat   2.0
g  snake   4.5
h     cat  NaN
i     dog   7.0
j     dog   3.0
```

Otra manera puede ser mediante el uso de doble corchete. En este caso no especificamos la cantidad de filas entonces nos muestra todas por defecto.

```
[ ]: # Otra manera

df[['animal', 'age']]
```

```
[ ]:  animal  age
      a    cat  2.5
      b    cat  3.0
      c  snake  0.5
      d    dog NaN
      e    dog  5.0
      f    cat  2.0
      g  snake  4.5
      h    cat NaN
      i    dog  7.0
      j    dog  3.0
```

7. Ahora seleccionemos las filas [3, 4, 8] y columnas ['animal', 'age'].

```
[ ]: df.loc[df.index[[3, 4, 8]], ['animal', 'age']]
```

```
[ ]:  animal  age
      d    dog NaN
      e    dog  5.0
      i    dog  7.0
```

8. Para filtrar los elementos de nuestra tabla lo que hacemos es poner una condición adentro de los corchetes. Por ejemplo, podemos seleccionar solamente las filas que tengan numero de visitas mayores que 3.

```
[ ]: df[df['visits'] > 3] #No hay ninguno ja
```

```
[ ]: Empty DataFrame
      Columns: [animal, age, visits, priority]
      Index: []
```

9. Seleccionamos las filas que le falten los datos de la edad, es decir que su valor sea NaN. En este caso la función `isnull()` se usa como condición que devuelve `True` si el valor de la columna es nulo o NaN.

```
[ ]: df[df['age'].isnull()]
```

```
[ ]:  animal  age  visits  priority
      d    dog  NaN      3      yes
      h    cat  NaN      1      yes
```

10. Podemos poner condiciones más complejas adentro de los corchetes. Seleccionamos las filas donde el animal sea cat y la edad es menor a 3.

```
[ ]: df[(df['animal'] == 'cat') & (df['age'] < 3)]
```

```
[ ]:  animal  age  visits  priority
      a    cat  2.5      1      yes
      f    cat  2.0      3      no
```

11. La función `between(a, b)` elige los elementos que se encuentren en el rango `[a,b]`. Sería el equivalente de hacer `columna >= a & columna <= b`. Seleccionamos las filas con edad entre 2 y 4 (inclusive).

```
[ ]: df[df['age'].between(2, 4)]
```

```
[ ]:  animal  age  visits  priority
a    cat   2.5      1      yes
b    cat   3.0      3      yes
f    cat   2.0      3      no
j    dog   3.0      1      no
```

14. Si queremos cambiar el valor de celdas específicas usamos el `=` como si fuera una variable normal. Cambiemos la edad en la fila 'f' a 1.5.

```
[ ]: df.loc['f', 'age'] = 1.5
```

15. La función `sum()` suma todos los valores de la columna seleccionada. Sumemos la cantidad de visitas de todas las filas de `df` (numero total de visitas).

```
[ ]: df['visits'].sum()
```

```
[ ]: 19
```

16. Calculemos la media de las edades de los animales de `df`. En este caso `groupby` agrupa las filas según el animal y con la función `mean` se calcula el promedio de la edad.

```
[ ]: df.groupby('animal')['age'].mean()
```

```
[ ]: animal
cat      2.333333
dog      5.000000
snake    2.500000
Name: age, dtype: float64
```

17. Agregemos una nueva fila 'k' a `df` con valores que queramos para cada columna.

```
[ ]: df.loc['k'] = ['dog', 5.5, 2, 'no']
df
```

```
[ ]:  animal  age  visits  priority
a    cat   2.5      1      yes
b    cat   3.0      3      yes
c  snake   0.5      2      no
d    dog  NaN      3      yes
e    dog   5.0      2      no
f    cat   1.5      3      no
g  snake   4.5      1      no
h    cat  NaN      1      yes
i    dog   7.0      2      no
```

j	dog	3.0	1	no
k	dog	5.5	2	no

Ahora borremos la columna nueva para volver a tener el Dataframe original. La función `drop` recibe el índice de la fila que se desea eliminar.

```
[ ]: df = df.drop('k')
df
```

```
[ ]:  animal  age  visits priority
a    cat   2.5      1      yes
b    cat   3.0      3      yes
c  snake   0.5      2      no
d    dog  NaN      3      yes
e    dog   5.0      2      no
f    cat   1.5      3      no
g  snake   4.5      1      no
h    cat  NaN      1      yes
i    dog   7.0      2      no
j    dog   3.0      1      no
```

18. La función `value_counts()` devuelve cuantas veces aparece cada valor único para una columna específica. Contemos que cantidad de animales hay en el `df`.

```
[ ]: df['animal'].value_counts()
```

```
[ ]: animal
cat      4
dog      4
snake    2
Name: count, dtype: int64
```

19. La función `sort_values(by, ascending)` ordena la tabla con el criterio que pidamos. En el parámetro `by` ponemos una lista con las columnas que queremos usar para ordenar y en `ascending` ponemos una lista de booleanos donde indicamos si queremos que el ordenamiento de la lista anterior sea ascendente o no. Entonces, ordenemos el `df` primero a partir de los valores de edad 'age' en orden *descendente*, luego por el valor de las visitas 'visits' en orden *ascendente* (la fila i debería estar primero y la d should be ultima).

```
[ ]: df.sort_values(by=['age', 'visits'], ascending=[False, True])
```

```
[ ]:  animal  age  visits priority
i    dog   7.0      2      no
e    dog   5.0      2      no
g  snake   4.5      1      no
j    dog   3.0      1      no
b    cat   3.0      3      yes
a    cat   2.5      1      yes
f    cat   1.5      3      no
```

c	snake	0.5	2	no
h	cat	NaN	1	yes
d	dog	NaN	3	yes

20. Con `map` podemos transformar los valores de columna entera, le pasamos un diccionario con elementos del tipo `{valor_viejo: valor_nuevo}`. La columna `'priority'` contiene valores `'yes'` y `'no'`. Vamos a reemplazarlos por: `'yes'` como `True` y `'no'` como `False`.

```
[ ]: df['priority'] = df['priority'].map({'yes': True, 'no': False})
df
```

```
[ ]:
  animal  age  visits  priority
a   cat   2.5      1     True
b   cat   3.0      3     True
c  snake  0.5      2    False
d   dog  NaN      3     True
e   dog   5.0      2    False
f   cat   1.5      3    False
g  snake  4.5      1    False
h   cat  NaN      1     True
i   dog   7.0      2    False
j   dog   3.0      1    False
```

21. En la columna `'animal'` cambiemos `'snake'` por `'python'`. Podemos usar `map` como antes o podemos usar `replace` que recibe primero el valor viejo y después el valor nuevo realizará el reemplazo.

```
[ ]: df['animal'] = df['animal'].replace('snake', 'python')
df
```

```
[ ]:
  animal  age  visits  priority
a   cat   2.5      1     True
b   cat   3.0      3     True
c  python 0.5      2    False
d   dog  NaN      3     True
e   dog   5.0      2    False
f   cat   1.5      3    False
g  python 4.5      1    False
h   cat  NaN      1     True
i   dog   7.0      2    False
j   dog   3.0      1    False
```